



Product Requirements Document

Gerenciador de finanças pessoais — aplicação web full-stack

Item	Definição
Produto	Financy
Versão	1.0
Status	Planejamento para desenvolvimento
Escopo	Desafio obrigatório — Back-end e Front-end
Arquitetura	API GraphQL + aplicação React
Banco de dados	SQLite com Prisma

Documento preparado a partir dos requisitos do desafio e das telas de referência.

1. Visão geral do produto

O Financy será uma aplicação web para organização de finanças pessoais. O sistema permitirá que cada usuário crie uma conta, autentique-se e gerencie suas próprias receitas, despesas e categorias financeiras.

A aplicação deverá apresentar um dashboard com o resumo financeiro do usuário, além de páginas específicas para gerenciamento de transações, categorias e dados da conta.

- API GraphQL desenvolvida em TypeScript;
- Banco de dados SQLite utilizando Prisma;
- Front-end React com TypeScript e Vite;
- Autenticação baseada em JWT;
- Interface baseada fielmente no layout fornecido no Figma.

2. Problema

Usuários precisam de uma forma simples e visual de acompanhar suas finanças pessoais, mas frequentemente encontram ferramentas complexas ou que não apresentam de maneira clara:

- Quanto possuem de saldo;
- Quanto receberam no mês;
- Quanto gastaram no mês;
- Em quais categorias estão gastando;
- Quais foram suas transações mais recentes.

O Financy busca centralizar essas informações em uma interface simples, organizada e objetiva.

3. Objetivo do produto

Permitir que o usuário acompanhe e organize suas finanças por meio do cadastro e gerenciamento de transações e categorias personalizadas.

3.1 Objetivos principais

- Disponibilizar cadastro e autenticação de usuários;
- Garantir isolamento total dos dados entre usuários;
- Permitir gerenciamento completo de transações;
- Permitir gerenciamento completo de categorias;
- Exibir indicadores financeiros no dashboard;
- Reproduzir fielmente a experiência apresentada no Figma;
- Disponibilizar o projeto para execução local.

4. Público-alvo

Pessoas que desejam organizar suas finanças pessoais de maneira simples, visual e centralizada.

4.1 Perfil principal

- Usuário individual;
- Possui receitas e despesas mensais;
- Deseja categorizar seus gastos;
- Precisa consultar rapidamente seu saldo e movimentações;
- Tem familiaridade básica com aplicações web.

5. Escopo da versão inicial

5.1 Funcionalidades obrigatórias

1. Cadastro de usuário;
2. Login;
3. Identificação do usuário autenticado;
4. Listagem de transações;
5. Criação de transação;
6. Edição de transação;
7. Exclusão de transação;
8. Listagem de categorias;
9. Criação de categoria;
10. Edição de categoria;
11. Exclusão de categoria;
12. Dashboard financeiro;
13. Página de perfil;
14. Logout;
15. Proteção das informações por usuário.

6. Tecnologias e restrições

Camada	Obrigatório	Flexível / recomendado
Back-end	TypeScript, GraphQL, Prisma, SQLite e CORS	Framework HTTP/GraphQL, JWT, bcrypt ou Argon2
Front-end	TypeScript, React, Vite sem framework e GraphQL	TailwindCSS, Shadcn/UI, Apollo Client, React Query, React Hook Form, Zod
Entrega	Repositório público com /backend e /frontend	Testes, CI/CD e deploy em branch separada

7. Requisitos funcionais

RF01 — Criar conta

O sistema deve permitir que uma pessoa crie uma nova conta.

Campos

- Nome completo;
- E-mail;
- Senha.

Regras

- O nome completo é obrigatório;
- O e-mail é obrigatório e deve possuir formato válido;
- Não pode existir mais de uma conta com o mesmo e-mail;
- A senha deve possuir no mínimo 8 caracteres;
- A senha nunca deve ser armazenada em texto puro;
- Após o cadastro, o usuário poderá ser autenticado automaticamente ou direcionado para o login.

Mensagens esperadas

- “Conta criada com sucesso”;
- “Este e-mail já está cadastrado”;
- “A senha deve ter no mínimo 8 caracteres”;
- “Preencha todos os campos obrigatórios”.

RF02 — Fazer login

O sistema deve permitir que um usuário cadastrado acesse sua conta.

Campos

- E-mail;
- Senha;
- Lembrar-me.

Regras

- E-mail e senha são obrigatórios;
- Credenciais inválidas não devem informar qual campo está incorreto;
- Após a autenticação, a API deve retornar um token JWT;
- O front-end deve armazenar e enviar o token nas operações autenticadas;
- Após o login, o usuário deve ser direcionado ao dashboard.

Mensagens esperadas

- “E-mail ou senha inválidos”;
- “Login realizado com sucesso”;
- “Sua sessão expirou. Entre novamente”.

RF03 — Controlar acesso aos dados

Cada usuário deve visualizar e manipular apenas os próprios dados.

Regras obrigatórias

- O identificador do usuário deve ser obtido pelo token JWT;
- A API não deve confiar em um userId enviado pelo front-end;
- Todas as consultas de categorias e transações devem ser filtradas pelo usuário autenticado;
- O usuário não pode editar ou excluir registros pertencentes a outra conta;
- Tentativas de acesso indevido devem retornar erro de autorização ou registro não encontrado.

RF04 — Visualizar dashboard

O dashboard deve apresentar um resumo financeiro da conta autenticada.

Indicadores

- Saldo total = total de receitas – total de despesas;
- Receitas do mês = soma das transações do tipo receita no mês atual;
- Despesas do mês = soma das transações do tipo despesa no mês atual.

Transações recentes

- Exibir as cinco transações mais recentes;
- Mostrar ícone da categoria, descrição, data, categoria, tipo e valor formatado.

Resumo de categorias

- Exibir nome, quantidade de transações e valor movimentado.

Ações

- Ver todas as transações;
- Gerenciar categorias;
- Criar nova transação;
- Navegar para perfil pela identificação do usuário.

RF05 — Listar transações

O sistema deve permitir que o usuário visualize todas as suas transações.

Informações da listagem

- Descrição;
- Data;
- Categoria;
- Tipo;
- Valor;
- Ação de editar;
- Ação de excluir.

Filtros

- Busca por descrição;
- Tipo: todos, receitas ou despesas;

- Categoria;
- Período mensal.

Paginação

- Exibir inicialmente 10 registros por página;
- Informar quantidade total de resultados;
- Permitir navegar entre páginas;
- Manter os filtros ao mudar de página.

Ordenação

- Por padrão, da transação mais recente para a mais antiga.

RF06 — Criar transação

O usuário deve conseguir registrar uma receita ou despesa por meio de um modal.

Campos

- Tipo: despesa ou receita;
- Descrição;
- Data;
- Valor;
- Categoria.

Regras

- Todos os campos são obrigatórios;
- O valor deve ser maior que zero;
- O valor deve ser armazenado sem perda de precisão monetária;
- A categoria deve pertencer ao usuário autenticado;
- O tipo deve aceitar apenas INCOME ou EXPENSE;
- Após a criação, listagens e indicadores devem ser atualizados.

Modal

- Título “Nova transação”;
- Alternador entre despesa e receita;
- Botão para fechar;
- Botão “Salvar”;
- Estado de carregamento durante o envio.

RF07 — Editar transação

O usuário deve conseguir editar uma transação própria.

Comportamento

- O mesmo modal de criação poderá ser reutilizado;
- Os campos devem ser preenchidos com os dados existentes;
- O título deve mudar para “Editar transação”;
- O usuário poderá alterar todos os campos;
- O sistema deve atualizar os indicadores após salvar.

RF08 — Excluir transação

O usuário deve conseguir excluir uma transação própria.

Comportamento

- Antes de excluir, o sistema deve solicitar confirmação;
- Após a confirmação, a transação deve ser removida;
- O dashboard deve ser recalculado;
- A listagem deve ser atualizada;
- Uma mensagem de sucesso deve ser apresentada.

Mensagem sugerida

- “Tem certeza de que deseja excluir esta transação? Esta ação não poderá ser desfeita.”

RF09 — Listar categorias

O sistema deve exibir todas as categorias criadas pelo usuário.

Indicadores

- Total de categorias;
- Total de transações;
- Categoria mais utilizada.

Dados de cada categoria

- Ícone;
- Nome;
- Descrição;
- Cor;
- Quantidade de transações;
- Ação de editar;
- Ação de excluir.

Categoria mais utilizada

- A categoria com maior quantidade de transações associadas;
- Em caso de empate, poderá ser exibida a categoria utilizada mais recentemente.

RF10 — Criar categoria

O usuário deve conseguir criar uma categoria por meio de um modal.

Campos

- Título;
- Descrição;
- Ícone;
- Cor.

Regras

- O título é obrigatório;
- A descrição é opcional;

- Um ícone deve ser selecionado;
- Uma cor deve ser selecionada;
- Recomenda-se impedir títulos duplicados dentro da mesma conta;
- A categoria criada deve pertencer automaticamente ao usuário autenticado.

Modal

- Título “Nova categoria”;
- Campo de título;
- Campo de descrição;
- Seletor visual de ícones;
- Seletor visual de cores;
- Botão “Salvar”;
- Botão para fechar.

RF11 — Editar categoria

O usuário deve conseguir editar uma categoria própria.

Comportamento

- Reutilizar o modal de categoria;
- Preencher os campos com os valores atuais;
- Permitir alterar título, descrição, ícone e cor;
- Atualizar todas as telas que exibem a categoria.

RF12 — Excluir categoria

O usuário deve conseguir excluir uma categoria própria.

Regra proposta

- Quando a categoria possuir transações associadas, a exclusão deve ser bloqueada para evitar perda ou inconsistência de dados.

Mensagem sugerida

- “Esta categoria possui transações associadas. Altere ou exclua essas transações antes de remover a categoria.”

Quando não houver vínculos

- Solicitar confirmação;
- Excluir a categoria;
- Atualizar a listagem e os indicadores.

RF13 — Visualizar perfil

O sistema deve apresentar uma página com as informações da conta.

Informações

- Avatar com iniciais do usuário;
- Nome;
- E-mail;

- Formulário de dados pessoais.

Regras

- O usuário poderá alterar o nome;
- O e-mail deverá ser exibido como somente leitura;
- O upload de avatar não faz parte da entrega obrigatória;
- O avatar padrão deverá usar as iniciais do usuário.

RF14 — Fazer logout

O usuário deve conseguir encerrar sua sessão.

Ao sair

- O token deve ser removido;
- Os dados armazenados da sessão devem ser limpos;
- O usuário deve ser direcionado para a tela de login.

8. Páginas da aplicação

Página 1 — Login

Rota sugerida: /login

O fluxo de recuperação de senha não faz parte dos requisitos obrigatórios. O link pode permanecer apenas como elemento visual na entrega principal.

- Logotipo Financy;
- Título “Fazer login”;
- Campo de e-mail;
- Campo de senha;
- Alternador de visibilidade da senha;
- Checkbox “Lembrar-me”;
- Link “Recuperar senha”;
- Botão “Entrar”;
- Separador visual;
- Botão “Criar conta”.

Página 2 — Criar conta

Rota sugerida: /register

- Logotipo;
- Nome completo;
- E-mail;
- Senha;
- Informação de senha mínima;
- Botão “Cadastrar”;
- Botão “Fazer login”.

Página 3 — Dashboard

Rota sugerida: /

Usuário deslogado: apresentar ou redirecionar para login. Usuário logado: apresentar o dashboard.

- Cabeçalho;
- Navegação;
- Cards financeiros;
- Transações recentes;
- Resumo de categorias;
- Botão para nova transação.

Página 4 — Transações

Rota sugerida: /transactions

- Título e subtítulo;
- Botão “Nova transação”;
- Filtros;
- Tabela;
- Paginação;
- Ações de edição e exclusão.

Página 5 — Categorias

Rota sugerida: /categories

- Título e subtítulo;
- Botão “Nova categoria”;
- Indicadores;
- Cards de categorias;
- Ações de edição e exclusão.

Página 6 — Perfil

Rota sugerida: /profile

- Avatar com iniciais;
- Nome;
- E-mail;
- Formulário para editar nome;
- Botão “Salvar alterações”;
- Botão “Sair da conta”.

9. Modais

9.1 Modal de transação

O mesmo componente deve ser utilizado para criação e edição.

- Variações: Nova transação e Editar transação.

- Estados: aberto, carregando, erro de validação, salvando e sucesso.

9.2 Modal de categoria

O mesmo componente deve ser utilizado para criação e edição.

- Variações: Nova categoria e Editar categoria.
- Estados: aberto, carregando, erro de validação, salvando e sucesso.

10. Fluxos principais

Fluxo de cadastro

Criar conta

- Preencher dados
- Validar formulário
- Enviar mutation GraphQL
- Criar usuário
- Autenticar ou redirecionar ao login
- Acessar dashboard

Fluxo de login

Informar e-mail e senha

- Validar credenciais
- Receber JWT
- Armazenar sessão
- Consultar usuário autenticado
- Acessar dashboard

Fluxo de criação de transação

Abrir modal

- Selecionar tipo
- Informar descrição, data, valor e categoria
- Salvar
- Atualizar listagem
- Recalcular dashboard
- Fechar modal

Fluxo de criação de categoria

Abrir modal

- Informar título e descrição
- Selecionar ícone e cor
- Salvar
- Atualizar categorias
- Fechar modal

11. Modelo de dados

11.1 User

Campo	Tipo	Regra
id	String/UUID	Chave primária
name	String	Obrigatório
email	String	Único e obrigatório
passwordHash	String	Obrigatório
avatarUrl	String	Opcional
createdAt	DateTime	Automático
updatedAt	DateTime	Automático

11.2 Category

Campo	Tipo	Regra
id	String/UUID	Chave primária
title	String	Obrigatório
description	String	Opcional
icon	String	Obrigatório
color	String	Obrigatório
userId	String	Proprietário
createdAt	DateTime	Automático
updatedAt	DateTime	Automático

11.3 Transaction

Campo	Tipo	Regra
id	String/UUID	Chave primária

Campo	Tipo	Regra
description	String	Obrigatório
type	Enum	INCOME ou EXPENSE
amount	Decimal	Maior que zero
date	DateTime	Obrigatório
categoryId	String	Categoria do usuário
userId	String	Proprietário
createdAt	DateTime	Automático
updatedAt	DateTime	Automático

11.4 Relacionamentos

User 1 — N Category
 User 1 — N Transaction
 Category 1 — N Transaction

12. Operações GraphQL

12.1 Queries

```
me: User!

dashboard: Dashboard!

transactions(
  filters: TransactionFiltersInput
  pagination: PaginationInput
): TransactionConnection!

transaction(id: ID!): Transaction

categories: [Category!]!

category(id: ID!): Category
```

12.2 Mutations de autenticação

```
register(input: RegisterInput!): AuthPayload!  
  
login(input: LoginInput!): AuthPayload!  
  
updateProfile(input: UpdateProfileInput!): User!
```

12.3 Mutations de transações

```
createTransaction(input: CreateTransactionInput!): Transaction!  
  
updateTransaction(id: ID!, input: UpdateTransactionInput!): Transaction!  
  
deleteTransaction(id: ID!): Boolean!
```

12.4 Mutations de categorias

```
createCategory(input: CreateCategoryInput!): Category!  
  
updateCategory(id: ID!, input: UpdateCategoryInput!): Category!  
  
deleteCategory(id: ID!): Boolean!
```

12.5 Dashboard

```
type Dashboard {  
  totalBalance: Decimal!  
  monthlyIncome: Decimal!  
  monthlyExpense: Decimal!  
  recentTransactions: [Transaction!]!  
  categorySummaries: [CategorySummary!]!  
}
```

13. Autenticação e autorização

A autenticação será baseada em JWT.

```
Authorization: Bearer <token>
```

13.1 Regras de segurança

- A senha deve ser processada com bcrypt ou Argon2;
- O JWT deve ser assinado com JWT_SECRET;
- Queries privadas devem exigir autenticação;
- O usuário autenticado deve ser obtido pelo contexto GraphQL;
- Toda edição e exclusão deve verificar a propriedade do registro;
- Senhas nunca devem ser retornadas pela API;
- Erros internos não devem expor stack trace em produção.

14. Variáveis de ambiente

14.1 Back-end — .env.example

```
JWT_SECRET=  
DATABASE_URL=  
FRONTEND_URL=  
PORT=
```

As duas primeiras variáveis são obrigatórias. As demais devem ser adicionadas ao exemplo caso sejam utilizadas.

14.2 Exemplo para SQLite

```
DATABASE_URL="file:./dev.db"
```

14.3 Front-end — .env.example

```
VITE_BACKEND_URL=
```

14.4 Exemplo de desenvolvimento

```
VITE_BACKEND_URL=http://localhost:3333/graphql
```

15. Requisitos não funcionais

RNF01 — Fidelidade visual

- Tipografia;
- Espaçamentos;
- Cores;
- Bordas;
- Sombras;
- Ícones;
- Dimensões;
- Estados dos componentes;
- Organização visual do Figma.

RNF02 — Responsividade

- A aplicação deve funcionar em desktop, notebook, tablet e dispositivos móveis;
- Em telas pequenas, tabelas podem utilizar rolagem horizontal ou representação em cards.

RNF03 — Experiência do usuário

- Indicador de carregamento;
- Mensagem de sucesso;
- Mensagem de erro;
- Estado vazio;
- Confirmação em ações destrutivas;

- Botões desabilitados durante envio.

RNF04 — Formatação

- Moeda: Real brasileiro;
- Formato: R\$ 1.250,00;
- Datas: DD/MM/AAAA;
- Valores de receita: sinal positivo;
- Valores de despesa: sinal negativo.

RNF05 — Desempenho

- Filtros e paginação devem ser executados pela API;
- A listagem não deve carregar todo o banco para paginar no front-end;
- O dashboard deve evitar consultas repetitivas desnecessárias;
- Após mutations, apenas as consultas afetadas devem ser atualizadas.

RNF06 — Acessibilidade

- Campos devem possuir labels;
- Botões de ícone devem possuir descrição acessível;
- Modais devem suportar teclado;
- O foco deve permanecer dentro do modal enquanto estiver aberto;
- Cores não devem ser o único indicativo de estado;
- Deve haver contraste adequado entre texto e fundo.

16. Estados da interface

Carregamento

Skeletons, spinner e botões com estado de processamento.

Lista vazia — Transações

Você ainda não possui transações. Registre sua primeira receita ou despesa.

Lista vazia — Categorias

Você ainda não possui categorias. Crie uma categoria para organizar suas transações.

Erro

Não foi possível carregar os dados. Tente novamente.

Sessão expirada

Sua sessão expirou. Entre novamente para continuar.

17. Códigos de erro recomendados

Código	Uso
UNAUTHENTICATED	Usuário não autenticado
FORBIDDEN	Operação não autorizada
BAD_USER_INPUT	Campos inválidos
NOT_FOUND	Registro não encontrado
CONFLICT	E-mail ou categoria duplicada
CATEGORY_IN_USE	Categoria possui transações
INTERNAL_SERVER_ERROR	Erro inesperado

18. Critérios de aceitação

Autenticação

- ☐ É possível criar uma conta;
- ☐ E-mails duplicados são bloqueados;
- ☐ É possível realizar login;
- ☐ Credenciais inválidas retornam erro;
- ☐ Rotas privadas não abrem sem autenticação;
- ☐ É possível fazer logout.

Isolamento de dados

- ☐ Um usuário não visualiza dados de outro;
- ☐ Um usuário não edita dados de outro;
- ☐ Um usuário não exclui dados de outro;
- ☐ O vínculo com o usuário é aplicado pela API.

Transações

- ☐ É possível criar uma transação;
- ☐ É possível editar uma transação;
- ☐ É possível excluir uma transação;
- ☐ É possível listar transações;
- ☐ Os filtros funcionam;

- ☐ A paginação funciona;
- ☐ Valores e datas são formatados corretamente.

Categorias

- ☐ É possível criar uma categoria;
- ☐ É possível editar uma categoria;
- ☐ É possível excluir uma categoria sem transações;
- ☐ A exclusão é protegida quando a categoria está em uso;
- ☐ É possível listar todas as categorias;
- ☐ Ícone e cor são apresentados corretamente.

Dashboard

- ☐ O saldo total é calculado corretamente;
- ☐ As receitas mensais são calculadas corretamente;
- ☐ As despesas mensais são calculadas corretamente;
- ☐ As transações recentes são exibidas;
- ☐ O resumo de categorias é exibido;
- ☐ Os valores são atualizados após mutations.

Entrega técnica

- ☐ O projeto utiliza TypeScript;
- ☐ A API utiliza GraphQL;
- ☐ O banco é acessado com Prisma;
- ☐ O banco utilizado é SQLite;
- ☐ O front-end utiliza React;
- ☐ O front-end utiliza Vite;
- ☐ O CORS está habilitado;
- ☐ Existem arquivos .env.example;
- ☐ O repositório está público;
- ☐ O projeto pode ser executado localmente.

19. Estrutura do repositório

```
financy/  
├── backend/  
│   ├── prisma/  
│   │   ├── schema.prisma  
│   │   └── migrations/  
│   ├── src/  
│   │   ├── auth/  
│   │   ├── users/  
│   │   └── transactions/
```

```
| | | categories/
| | | dashboard/
| | | graphql/
| | | shared/
| | .env.example
| | package.json
| | README.md
| frontend/
| | src/
| | | components/
| | | pages/
| | | layouts/
| | | graphql/
| | | hooks/
| | | contexts/
| | | services/
| | | schemas/
| | | utils/
| | | styles/
| | .env.example
| | package.json
| | README.md
| README.md
```

20. Priorização

P0 — Obrigatório para entrega

- Cadastro;
- Login;
- JWT;
- Isolamento dos dados;
- CRUD de transações;
- CRUD de categorias;
- Dashboard;
- React;
- Vite;
- GraphQL;
- Prisma;
- SQLite;
- TypeScript;
- CORS;
- .env.example;
- Repositório público com backend e frontend.

P1 — Importante para fidelidade ao Figma

- Perfil;
- Alteração de nome;
- Filtros de transações;
- Paginação;
- Seleção de cor e ícone;
- Confirmações de exclusão;
- Estados vazios e de carregamento;
- Responsividade.

P2 — Funcionalidades extras

- Upload de avatar;
- Recuperação de senha;
- Refresh token;
- Modo escuro;
- Exportação de transações;
- Relatórios e gráficos;
- Orçamento mensal;
- Categorias padrão;
- Testes end-to-end ampliados;
- Deploy automatizado.

Funcionalidades P2 devem ser implementadas somente após a entrega obrigatória, preferencialmente em outra branch.

21. Fora do escopo da entrega principal

- Integração bancária;
- Compartilhamento de conta;
- Múltiplas moedas;
- Parcelamento;
- Transações recorrentes;
- Notificações;
- Recuperação real de senha;
- Upload de comprovantes;
- Aplicativo mobile nativo;
- Controle de cartões;
- Metas financeiras;
- Upload de avatar.

22. Plano de implementação

Etapa 1 — Fundação

- Criar monorepositório;
- Criar pastas backend e frontend;
- Configurar TypeScript;
- Configurar .env.example;
- Configurar Prisma e SQLite;
- Preparar Style Guide do front-end.

Etapa 2 — Autenticação

- Modelar usuário;
- Criar cadastro;
- Criar login;
- Gerar JWT;
- Criar contexto de autenticação;
- Implementar rotas protegidas.

Etapa 3 — Categorias

- Modelar categoria;
- Implementar queries e mutations;
- Criar página;
- Criar modal;
- Implementar edição e exclusão.

Etapa 4 — Transações

- Modelar transação;
- Implementar queries e mutations;
- Criar filtros e paginação;
- Criar modal;
- Implementar edição e exclusão.

Etapa 5 — Dashboard

- Calcular saldo;
- Calcular receitas e despesas mensais;
- Buscar transações recentes;
- Gerar resumo de categorias;
- Integrar os cards e atalhos.

Etapa 6 — Perfil e acabamento

- Criar página de perfil;
- Implementar alteração de nome;
- Implementar logout;

- Adicionar feedbacks;
- Revisar responsividade;
- Revisar fidelidade ao Figma.

Etapas 7 — Qualidade e entrega

- Testar isolamento de dados;
- Testar todas as mutations;
- Validar `.env.example`;
- Criar migrations;
- Criar README;
- Remover mocks;
- Publicar repositório;
- Validar instalação do zero.

23. Definição de pronto

1. Todas as funcionalidades obrigatórias estiverem implementadas;
2. O front-end consumir exclusivamente a API GraphQL;
3. Não houver dados simulados nas telas finais;
4. Cada usuário acessar somente os próprios dados;
5. O projeto puder ser iniciado localmente seguindo o README;
6. As migrations do Prisma estiverem disponíveis;
7. Os arquivos `.env.example` estiverem atualizados;
8. A interface estiver visualmente próxima do Figma;
9. O repositório estiver público;
10. As pastas backend e frontend estiverem na raiz;
11. O código obrigatório estiver preservado na branch enviada para avaliação;
12. Funcionalidades extras, quando existentes, estiverem em uma branch separada.

— Fim do documento —